

IMPACT ANALYSIS

CR-001: Student Active/Inactive Status

1. Change Summary

CR-001 introduces an Active/Inactive status for students in the Student Management System.

Administrators will be able to deactivate and reactivate students without deleting their records. Inactive students will not be able to log in, and students who are deactivated while logged in will lose access on their next protected request.

The change requires a database schema change, a new administrator-only API endpoint, changes to login and student profile behaviour, and updates to the administrator and student dashboards.

2. Impact Assessment

Area	Impact	Assessment
Frontend	Moderate	Status column, badge, Activate/Deactivate action, status filter, and inactive-session handling will be added.
Backend	Moderate - High	New status endpoint, login status check, and server-side rejection of existing tokens of inactive students.
API	Moderate	One new endpoint, status added to student responses, and changed login and /me behaviour.
Database	High	New students.status column with migration, rollback, schema.sql, and seed.sql changes.
Authentication	Moderate	Login must reject inactive students. Token handling must also reject them.
Authorization	Low	New endpoint reuses existing ADMIN protection. No role changes.
Performance	Low	One extra student lookup may be added to login and protected student requests.
CRUD	Moderate	Add must default to ACTIVE. Edit must not change status. Edit/Delete must use correct student_id.
Data Migration	Moderate	Existing students must become ACTIVE with no data loss.
CR-001 Search	Low - Moderate	Status filter must work together with the existing search and result count.
Deployment	High	Migration must run before the new backend. A deployment order and rollback plan are required.

3. Frontend Impact

The main affected areas are the administrator dashboard and the student dashboard.

Potential changes include:

- Status column and Active/Inactive badge in the student table.
- Activate/Deactivate action with confirmation.
- Table refresh and error message after a status change.
- Status filter (All, Active, Inactive).

- Combined behaviour of status filter, search, and result count.
- Handling a 403 response on the student dashboard by ending the session.
- Responsive styling for the new column, action, and filter.

Potentially affected files:

- `frontend/src/pages/DashboardPage.jsx`
- `frontend/src/pages/StudentDashboardPage.jsx`
- `frontend/src/components/StudentStatusBadge.jsx` (optional)
- Existing dashboard stylesheet

The implementation must preserve the relationship between each displayed student and its original `student_id`, including when the list is filtered by search or by status.

When the status filter is set to Active or Inactive, a student may leave the visible list immediately after the status is changed. The result count must update accordingly.

`LoginPage.jsx` already displays the message returned by the server, so no change is expected there. This must be verified during testing.

4. Backend, API and Database Impact

4.1 Database Impact

- A `status` column (`ENUM('ACTIVE', 'INACTIVE') NOT NULL DEFAULT 'ACTIVE'`) will be added to the `students` table.
- The `users` table, existing columns, constraints, and the foreign key remain unchanged.
- `schema.sql` uses `CREATE TABLE IF NOT EXISTS`, so it will not update an existing database. A separate migration script and a rollback script are required.
- `seed.sql` must be updated. An `INACTIVE` sample student needs its own `users` row and `students` row.
- `database/README.md` must be updated, including the database version.
- No new index is required at the current data size.

4.2 Backend Impact

- Student model queries must return `status`. The existing update query lists its columns explicitly and must not include `status`.
- New students must be created as `ACTIVE`.
- A new function is required to update the status of a student by `student_id`.
- Login must check the status of `STUDENT` users after the password is verified. Administrator users have no student record and must not be blocked.
- The JWT contains only `userId`, `username`, and `role`, so the status cannot be taken from the token. It must be read from the database.
- Existing tokens of inactive students must be rejected. This can be done in the `authenticate` middleware or in the student-protected handler. The chosen approach must be justified in the Implementation Report.

- The new route must use `authenticate` and `authorizeRoles("ADMIN")`.

4.3 API Impact

Endpoint	Impact	Assessment
POST /api/auth/login	Moderate	Inactive students are rejected with 403 after the credentials are verified.
GET /api/students/	Low	status is added to each student record.
GET /api/students/:id	Low	status is added to the response.
GET /api/students/me	Moderate	status is added. Inactive students are rejected with 403.
PATCH /api/students/:id/status	New	Administrator only. Accepts ACTIVE or INACTIVE. Returns 400 for an invalid value and 404 for an unknown student.
POST /api/students/	Low	New student is created as ACTIVE.
PUT /api/students/:id	None	Must not change status.
DELETE /api/students/:id	None	Behaviour remains unchanged.

Adding the `status` field to existing responses does not break the current frontend, which ignores fields it does not use.

4.4 Deployment Order

The change must be deployed in this order:

1. Back up the database.
2. Run the migration script.
3. Deploy the backend.
4. Deploy the frontend.

If the backend is deployed before the migration, student queries will fail because the `status` column does not exist. If the frontend is deployed before the backend, the status column and actions will not work.

For rollback, the application code and the database must be rolled back together. The rollback script removes the status data, so students who were inactive will regain access.

5. Security Impact

The existing authentication and authorization mechanisms remain in place, but login and student request handling will include a status check.

The student list and the new status endpoint must be accessible only to authorized administrators.

The change must:

- Restrict the status endpoint to authenticated administrators.
- Enforce the inactive restriction on the backend, not only in the frontend.
- Reject previously issued tokens of inactive students.
- Show the inactive-account message only after the credentials are verified.
- Validate the status value on the server and use parameterised queries.

- Not allow a student to change any status, including their own.
- Not block administrator login.
- Not expose additional sensitive information.
- Use normal React rendering.

The default token lifetime is 1 day. If the status were checked only at login, a deactivated student could keep access for up to one day. This is the main security consideration of this CR.

6. CRUD and Functional Impact

Existing administrator CRUD operations must continue to work after the change.

Particular attention is required for:

- Creating a student: the status must default to ACTIVE.
- Editing a student, including an inactive student, without changing the status.
- Deleting a student, including an inactive student.
- Changing a status from a list filtered by search or by status.
- Maintaining the correct `student_id`.
- Login and profile viewing for active students.
- A reactivated student logging in again with the same credentials and unchanged data.
- A STUDENT user without a matching student record: the login behaviour must be defined and tested.

Changing a status must update only the status of the selected student. It must not change the student identity, personal data, or the linked user account.

7. Performance Impact

The expected performance impact is low.

- The additional column adds negligible storage.
- Login, and protected student requests depending on the chosen approach, add one student lookup by `user_id`. This column is unique and therefore indexed.
- If the status is checked in the `authenticate` middleware, the check must apply only to STUDENT users so that administrator requests are not affected.
- The status filter and result count operate on already-loaded records, as in CR-001. No API request is generated when the filter changes.
- A status change generates one PATCH request and one refresh of the student list.

This approach is suitable for the current expected dataset size. A large number of students may require server-side filtering and pagination in a future change.

8. Testing Impact

Manual testing should cover:

Database

- Migration on an existing database that contains data (existing students become ACTIVE).
- Fresh installation using `schema.sql` and `seed.sql`.
- Rollback script.
- Invalid status value rejected by the database.

API (Postman)

- Change status as administrator (ACTIVE to INACTIVE and back).
- Change status as a student and without a token.
- Invalid status value and unknown student ID.
- Login as an active student, an inactive student, an administrator, and with a wrong password.
- `GET /api/students/me` using a token issued before the deactivation.

Frontend

- Status displayed for every student.
- Deactivate and Activate, including cancelling the confirmation.
- Error message when a status change fails.
- Status filter alone and combined with search, and the result count.
- Inactive-account message on the login page.
- Student deactivated while logged in: session ends and the login page is shown.

Regression

- Add, Edit, and Delete student, including Edit and Delete from filtered results with the correct `student_id`.
- CR-001 search, Clear Search, and no-result state.
- Administrator login and authorization.
- Student profile viewing.
- Responsive layout of the dashboard.

9. Risk Analysis

Risk	Impact	Mitigation
Inactive student keeps access through an existing token	High	Check the status on the server for protected student requests, not only at login. Test with an old token.
Administrator login blocked by the new status check	High	Apply the check only to STUDENT users. Test administrator login.
Migration fails or damages existing data	High	Back up first, test on a copy, use DEFAULT 'ACTIVE', and compare record counts before and after.
Wrong student status changed, especially from a filtered list	High	Use the original <code>student_id</code> . Test from searched and filtered lists.

Risk	Impact	Mitigation
Status endpoint accessible to non-administrators	High	Use authenticate and authorizeRoles("ADMIN"). Test with a student token and without a token.
Existing CRUD or CR-001 search breaks	High	Perform regression testing.
Backend deployed before migration (unknown column status)	Medium - High	Follow the deployment order. Document the migration in database/README.md.
Rollback re-enables deactivated students	Medium - High	Record the inactive students before rollback and roll back code and database together.
Invalid status value saved	Medium	Validate on the server and use the ENUM constraint.
Inactive message reveals account status to others	Low - Medium	Show the message only after correct credentials.
STUDENT user without a student record handled unclearly	Low - Medium	Define the behaviour and test it.
Responsive UI problems	Low	Test supported screen sizes.

10. Overall Impact and Recommendation

Overall Impact: High

CR-001 is a full-stack change. It requires a database schema change, a new administrator-only API endpoint, changes to login and student request handling, and updates to both dashboards. Existing data, user roles, and authorization rules are preserved.

The change is technically feasible and recommended for implementation, subject to formal approval, a database backup before migration, and subsequent testing.

Key implementation considerations: Deactivation must be enforced on the server for both login and previously issued tokens. Administrator login must not be affected. The migration must be applied before the new backend is deployed, and a rollback plan must be available.